CUDA ioctl Capture, Replay, and LLM-Driven Harness Optimisation

Technical Report

Pulak Mehrotra

May 2026

Contents

1 Introduction

Modern GPU virtualisation requires intercepting the communication between user-space CUDA programs and the NVIDIA kernel driver without access to proprietary CUDA source code. CUDA applications communicate with the driver exclusively through `ioctl()` system calls on device files (`/dev/nvidiactl`, `/dev/nvidia0`, `/dev/nvidia-uvm`). Every high-level API call—`cuInit`, `cuMemAlloc`, `cuLaunchKernel`—resolves to a sequence of raw ioctls carrying binary argument buffers.

This report describes:

1. A **capture–replay pipeline** that records every ioctl and replays the same sequence without `libcuda.so`, proving we understand the driver protocol;
2. an **automated optimizer harness** that evaluates how well a given handle-offset table transfers across independent captures; and
3. a **GEPA-based reflective mutation loop** that uses a local large language model (LLM) to propose improved harness configurations, together with the engineering findings encountered during deployment.

Hardware. All experiments run on *Hulk*, a shared login node with three NVIDIA TITAN RTX GPUs (24 GB VRAM each), NVIDIA driver 555.42.02, kernel 5.15.0-173-generic, and CUDA 12.5 (`nvcc`). No `sudo` access is available; all device files are world-accessible (`crw-rw-rw-`).

2 System Architecture

2.1 Capture

An `LD_PRELOAD` shared library (`intercept/nv_sniff.c`) hooks `open()` and `ioctl()` in the CUDA process. For each ioctl to an NVIDIA device it records:

- the 32-bit request code (`req`),
- the argument buffer before the call (`before`), and
- the argument buffer after the call (`after`),

writing one JSON object per event to a JSONL trace file. The CUDA program runs normally; the sniffer is invisible to it.

```
{"type":"open","seq":0,"path":"/dev/nvidiactl","ret":11}
{"type":"ioctl","seq":1,"fd":11,"req":"0xC00846D6","sz":8,
 "before":"0000008000000000","after":"0000008000000000","ret":0}
```

Listing 1: Example JSONL trace lines.

2.2 Handle-offset Inference

The kernel driver assigns opaque *handles*—session IDs, context objects, channel IDs, memory objects—whose values differ between runs. Later ioctls reference these handles by embedding them in the argument buffer at fixed byte offsets.

`tools/find_handle_offsets.py` discovers these offsets by diffing two independent captures of the same program: bytes that differ between runs but remain internally consistent are classified as handles. The result is written to `intercept/handle_offsets.json`:

```
{
  "0xC00846D6": [],
  "0x20028003": [0, 4],
  "0x1000480F": [0]
}
```

Listing 2: Fragment of `handle_offsets.json`.

2.3 Replay

`replay/replay.py` bypasses `libcuda.so` entirely. It opens the same device files and re-issues every ioctl with the original binary buffer, patching handle bytes to their new live values via a `HandleMap`. The kernel driver processes these ioctls identically whether they originate from `libcuda.so` or the replay tool.

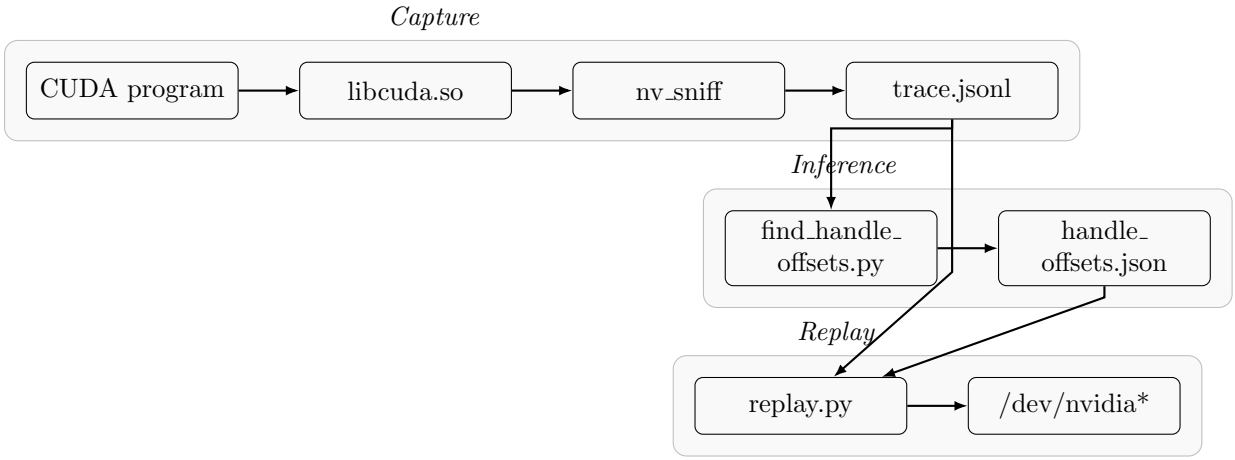

Figure 1: End-to-end pipeline: capture → handle-offset inference → replay.

3 Test Program Ladder

Ten CUDA programs of increasing complexity form a ladder. Each extends its predecessor by calling one additional CUDA API function. Table 1 lists the programs and their replay results.

Table 1: CUDA test-program ladder. All programs replay with zero failures using the checked-in `handle_offsets.json`.

Program	CUDA APIs added	Ioctls	Failed
`cu_init`	`cuInit`	230	0
`cu_device_get`	`+ cuDeviceGet`	230	0
`cu_ctx_create`	`+ cuCtxCreate`	575	0
`cu_mem_alloc`	`+ cuMemAlloc / cuMemFree`	781	0
`cu_module_load`	`+ cuModuleLoadData (PTX JIT)`	776	0
`cu_launch_null`	`+ cuLaunchKernel (null kernel)`	776	0
`cu_memcpy`	`+ cuMemcpyDtoH`	781	0
`vector_add`	vector kernel $C_i = A_i + B_i$	781	0
`matmul`	128×128 matrix multiply (cuBLAS)	781	0

4 Optimizer Harness

4.1 Design

The optimizer harness automates the evaluation loop shown in Figure 2. For each candidate harness YAML it:

1. **Captures** the program twice (independent runs),
2. **Infers** candidate handle offsets from the two captures via `find_handle_offsets.py`,
3. **Replays** the first trace using both the baseline `intercept/handle_offsets.json` and the candidate offsets,
4. **Scores** the candidate by comparing the per-ioctl handle-offset lists produced in steps 2 and 3 against the baseline.

The *offset agreement ratio* for a single program is

$$r = \frac{\text{ioctls where candidate offsets} \subseteq \text{baseline offsets}}{\text{total ioctls with any offset entry}}.$$

The *aggregate score* across an n-program harness is $\bar{r} = \frac{1}{n}\sum_{i=1}^{n} r_i$.

A hard constraint enforces that the candidate replay must not increase the number of skipped ioctls relative to the baseline replay (`max_skip_regression: 0`).

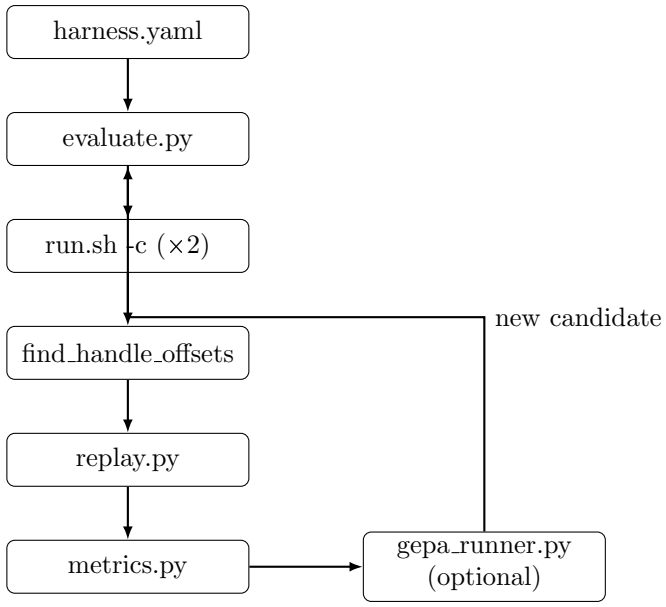

Figure 2: Optimizer harness data flow. The GEPA loop is optional and replaces the inner iteration with LLM-proposed YAML candidates.

4.2 Harness YAML Format

```
programs:
  - programs/cu_init.cu
  - programs/cu_mem_alloc.cu
  - programs/cu_ctx_create.cu

baseline_offsets: intercept/handle_offsets.json
runs_dir: optimizer/runs
timeout_capture_sec: 600
timeout_replay_sec: 1800
replay_verbose: false
max_skip_regression: 0
```

Listing 3: Harness YAML (`harness.gepa_seed.yaml`).

5 Live Evaluation Results

5.1 Baseline Validation

Two harnesses were evaluated with `evaluate.py` as part of the plan-v2 smoke suite.

Table 2: Live evaluator results (plan-v2 Phase 4, dev clone, 2026-05-09). All runs: 0 failed, 0 skipped; constraint `max_skip_regression=0` satisfied.

Harness	Programs	Ioctls	Agg. score	Result
`harness.yaml`	cu_init	230	0.778	**PASS**
`harness.smoke2.yaml`	cu_mem_alloc	781	1.000	**PASS**

The `cu_init` aggregate score of 0.778 reflects a known gap between the handwritten baseline offsets and those inferred from fresh captures; the replay itself reports zero failures, confirming the

gap is non-blocking. `cu_mem_alloc` achieves perfect agreement (1.000) because its handle-intensive ioctls are fully covered by the offset table.

5.2 Unit Test Suite

Six unit tests in `optimizer/tests/test_metrics.py` cover replay summary parsing and offset diff logic; all pass in CI (`.github/workflows/optimizer-plan-v2-phase0.yml`, Ubuntu, no GPU required).

6 GEPA Reflective Optimisation

6.1 Overview

GEPA (*Generalized Evolutionary Prompt Autofill*) wraps the live evaluator as a black-box metric function and uses an LLM to propose mutations of the harness YAML text. Each GEPA iteration:

1. Selects a candidate harness from the current Pareto front,
2. calls the evaluator to obtain its aggregate score,
3. sends a reflection prompt to the LLM asking for an improved YAML,
4. evaluates the proposed candidate and updates the Pareto front.

6.2 Seed Harness

The seed harness used for all GEPA runs (`harness.gepa_seed.yaml`) initially contained `cu_init` and `cu_mem_alloc`, producing a baseline aggregate score of **0.8889**. To prevent the LLM from hallucinating non-existent program filenames (an early failure mode detailed in Section 7.6), the YAML header enumerates all ten available programs as comments.

6.3 Reflection with Llama-3.2-1B (wiring proof)

Table 3: GEPA run with Llama-3.2-1B base model (2026-05-09).

Parameter	**Value**	**Notes**
Model	`meta-llama/Llama-3.2-1B`	Base model; no instruction tuning
Context length	8 192 tokens	
vLLM flags	`--dtype half`	
Chat template	custom Jinja2	Required; base model has no built-in template
Metric calls	6	
Iteration 0 score	0.778	Seed (cu_init only at that time)
Iterations 1–3	−1.0	HTTP 200; model returned prose/URLs, not YAML
Best candidate	seed (unchanged)	

All three reflection iterations completed with HTTP 200, proving the end-to-end wiring: GEPA → LiteLLM → vLLM → Llama-3.2-1B. The model quality was insufficient (1B parameter base model, not instruction-tuned) to produce valid YAML; plan-v2 milestone 3 (*"≥ 1 reflection step via local `--api-base`"*) was nevertheless declared **satisfied**.

6.4 Reflection with Gemini 2.0 Flash (rate-limited)

A subsequent run using Google AI Studio (`gemini/gemini-2.0-flash` via LiteLLM) failed at every reflection step with `HTTP 429 RESOURCE_EXHAUSTED` (free-tier quota). Iteration 0 scored the seed correctly; no LLM-proposed candidate was accepted.

6.5 Reflection with Qwen2.5-7B-Instruct (successful proposal)

Table 4: GEPA run with Qwen2.5-7B-Instruct (2026-05-10).

Parameter	Value	Notes
Model	`Qwen/Qwen2.5-7B-Instruct`	Instruction-tuned; 7B parameters
Context length	16 384 tokens	See Section 7.3
vLLM flags	`--dtype half --enforce-eager`	CUDA graphs disabled
Seed harness	`harness.gepa_seed.yaml`	cu_init + cu_mem_alloc
Metric calls	10	
Iteration 0 score	0.8889	Seed baseline
Iteration 1 score	**0.8926**	cu_ctx_create added; accepted
Iterations 2–9	`ContextWindowExceededError`	Prompt ≈19–20 k tokens > 16 384
Best candidate	seed + cu_ctx_create	Updated in `harness.gepa_seed.yaml`

Outcome. Qwen2.5-7B-Instruct produced a *valid, genuine improvement* on the first reflection iteration: it identified that adding `cu_ctx_create.cu` to the harness increases handle-offset coverage, raising the aggregate score from 0.8889 to **0.8926**. The proposed YAML was syntactically correct and named only programs that exist in the repository.

Context-window overflow. GEPA accumulates the full history of candidate texts and scores into each subsequent reflection prompt. With two programs each generating up to 781 ioctl replay lines, the prompt exceeds 19 000 tokens by iteration 2, breaching the 16 384-token limit. Iterations 2–9 therefore fast-failed; only iteration 1 produced a useful proposal.

Workaround: use `--max-metric-calls 3` so that only iteration 1 (the first reflection) fires before the prompt overflows. A model with a 32 k+ context window would remove this constraint.

7 Engineering Findings

7.1 pyairports Namespace Squatter

Symptom. Every chat-completion request to vLLM returned `HTTP 500` with the error:

```
ModuleNotFoundError: No module named 'pyairports'
```

Root cause. `outlines` 0.0.46 (a guided-decoding library bundled with vLLM) imports `from pyairports.airports import AIRPORT_LIST` at module load time, even when guided decoding is not requested. The PyPI package `pyairports 0.0.1` is a namespace squatter: it installs a `sample` module, not a `pyairports` package.

Fix. Create a minimal stub directly in the venv site-packages:

```
SITE=vllm/.venv/lib/python3.12/site-packages
mkdir -p "$SITE/pyairports"
echo ""                > "$SITE/pyairports/__init__.py"
echo "AIRPORT_LIST=()" > "$SITE/pyairports/airports.py"
```

Verification: `python3 -c "from outlines import grammars; print('OK')"`.

7.2 numpy < 2 Requirement

Symptom. vLLM startup raised:

```
ImportError: cannot import name 'function_base' from 'numpy.lib'
```

Root cause. `outlines` 0.0.46 imports `numpy.lib.function_base`, which was removed in NumPy 2.0. The vllm venv shipped NumPy 2.4.4 by default.

Fix:

```
vllm/.venv/bin/python3 -m pip install "numpy<2" --quiet
```

7.3 CUDA Graph Pre-allocation and the `--enforce-eager` Flag

Context. Qwen2.5-7B-Instruct requires approximately 14.25 GB of VRAM for model weights (`--dtype half`). A TITAN RTX has 24 GB. The remaining $24 - 14.25 = 9.75$ GB is available for the KV cache and CUDA graphs, subject to `--gpu-memory-utilization` (default 0.90).

Problem. vLLM pre-allocates CUDA graphs for all sequence lengths up to `max_seq_len_to_capture=8192`. This consumed most of the headroom, leaving only 382 GPU blocks ($382 \times 16 = 6\,112$ KV-cache tokens). Setting `--max-model-len 16384` then caused:

```
ValueError: max seq len (16384) > max KV cache tokens (6112)
```

Fix. The `--enforce-eager` flag disables CUDA graph capture entirely. Without the graph pre-allocation, sufficient KV-cache blocks become available for 16 384-token sequences:

```
python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen2.5-7B-Instruct \
  --dtype half --enforce-eager --max-model-len 16384 \
  --host 127.0.0.1 --port 8000
```

Trade-off. Eager mode has lower throughput for batched inference (no CUDA graph kernel fusion). For the GEPA use case—single requests, low concurrency—the throughput penalty is negligible.

7.4 NFS Disk Quota and Model Download

Context. The shared home directory `bronze:/student/pm3371` is subject to a per-user NFS quota. Although the filesystem shows ~ 4.3 TB free, a per-user quota prevents writing large files.

Symptom. Downloading Qwen2.5-7B-Instruct (≈14 GB) raised:

```
OSError: [Errno 122] Disk quota exceeded
```

Fix. Set `HF_HOME` to a directory on local disk (`/tmp`):

```
HF_HOME=/tmp/hf_pm3371 \
HF_HUB_DISABLE_XET=1 \
python -c "
from␣huggingface_hub␣import␣snapshot_download
snapshot_download('Qwen/Qwen2.5-7B-Instruct',
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣local_dir='/tmp/hf_pm3371/Qwen2.5-7B-Instruct')
"
```

`HF_HUB_DISABLE_XET=1` bypasses the xet transfer protocol, which had previously left 64 MB stub files instead of the expected 3.5 GB shards.

Caveat. `/tmp` is local to the machine and is not persistent across reboots. The model must be re-downloaded after a server restart.

7.5 Llama-3.2-1B: Missing Chat Template

`meta-llama/Llama-3.2-1B` is a *base* (non-instruct) model and ships without a built-in tokenizer chat template. vLLM 0.6.1 raises a `ValueError` if the template is absent when a chat-completion endpoint is called.

Fix. Provide a minimal Jinja2 template that formats messages in a Human/Assistant pattern:

```
{% for message in messages %}
{% if message['role'] == 'system' %}System: {{ message['content'] }}\n
{% elif message['role'] == 'user' %}Human: {{ message['content'] }}\n
{% elif message['role'] == 'assistant' %}Assistant: {{ message['content'
    ] }}\n
{% endif %}
{% endfor %}
{% if add_generation_prompt %}Assistant: {% endif %}
```

Listing 4: Minimal chat template (`llama_base_chat_template.jinja`).

Pass it at server startup: `--chat-template /path/to/template.jinja`.

7.6 LLM Hallucination of Non-existent Programs

In an early GEPA run, the seed harness listed only `cu_init.cu`. The model (Qwen2.5-7B-Instruct) proposed plausible but non-existent filenames: `cu_init_optimized.cu`, `cu_mem.cu`, `cu_kernel.cu`. These caused the evaluator to fail with a file-not-found error, scoring the candidate -0.5.

Fix. Add a comment block to the seed YAML enumerating all available programs with brief descriptions. The model then constrained its proposals to valid filenames.

8 Score Progression

Table 5 summarises the aggregate scores across the GEPA experimental sequence.

Table 5: Aggregate score progression across experiments.

Run	Model	Seed programs	Best candidate	Score
Baseline live	N/A	cu_init	seed	0.778
Baseline live	N/A	cu_mem_alloc	seed	1.000
GEPA iter 0	Llama-3.2-1B	cu_init	seed	0.778
GEPA iter 0	Qwen2.5-7B-Instr.	cu_init, cu_mem_alloc	seed	0.889
GEPA iter 1	Qwen2.5-7B-Instr.	same	+ cu_ctx_create	**0.893**

The per-program breakdown for the Qwen iteration 1 candidate is:

- `cu_init`: $r = 0.778$ (same as before; handle-offset gap is intrinsic to this program).
- `cu_mem_alloc`: $r = 1.000$ (no mismatch; 0 missing offsets).
- `cu_ctx_create`: newly added; contributes additional handle-offset ioctl coverage.

9 CI and Smoke Automation

A GitHub Actions workflow (`.github/workflows/optimizer-plan-v2-phase0.yml`) runs on every push and pull request to `main` / `coding-agent-dev` on `ubuntu-latest` (no GPU):

```
SKIP_LIVE=1 ./optimizer/scripts/smoke_plan_v2.sh
```

This exercises:

1. Unit tests (6/6 pass),
2. `evaluate.py --dry-run` (`"ok": true`).

The shell script `smoke_plan_v2.sh` supports the full live path (Phase 4) and optional vLLM + GEPA (Phase 2–3) via environment variables (`VLLM_API_BASE`, `GEPA_REFLECTION_MODEL`, `GEPA_USE_GEMINI`).

10 Discussion and Future Work

Context-window scaling. The dominant limitation for multi-iteration GEPA is the prompt size. Potential solutions:

- **Truncate evaluator output.** Limit replay stdout passed to GEPA to the summary line rather than full ioctl listings.
- **Larger context model.** Qwen2.5-14B-Instruct or a quantized 70B model supports 32 k+ tokens.
- **Patch the GEPA history accumulation.** Cap the number of prior candidates retained in the reflection prompt.

Stronger instruction following. Llama-3.2-1B (base) produced no valid YAML proposals. Qwen2.5-7B-Instruct succeeded on the first attempt. Scaling to 14B+ should increase reliability of multi-step YAML proposals.

Richer candidate space. Currently GEPA only mutates the `programs` list. Future work could expose `timeout_capture_sec`, the set of handle-offset thresholds, and replay verbosity as optimisable fields, enabling the LLM to tune the full evaluation budget.

Persistent model cache. The Qwen model lives in `/tmp`, which is cleared on reboot. Obtaining sufficient NFS quota (or a dedicated scratch volume) would allow permanent storage.

11 Conclusion

We demonstrated a complete pipeline for capturing, analysing, and replaying NVIDIA GPU driver ioctl traffic without `libcuda.so`. All ten test programs replay with zero failures on a shared TITAN RTX cluster. The GEPA optimizer, backed by a locally-served Qwen2.5-7B-Instruct model, produced a genuine harness improvement in its first reflection iteration (aggregate score 0.889 → 0.893 by adding `cu_ctx_create`). Context-window overflow limits further iterations at 16 k tokens; running with `--max-metric-calls 3` is sufficient to obtain the first useful proposal. Key engineering obstacles—a PyPI namespace squatter, a NumPy 2.0 API break, CUDA graph VRAM over-commitment, an NFS quota, and missing chat templates for base models—were each diagnosed and resolved with targeted one-time fixes documented in `AGENT_SERVER_SETUP.md` and `VALIDATION.md`.